

ICML 2023 | 换一个对偶视角，Kernel PCA 同时变快、变稳、变稀疏

KU Leuven 提出把 Kernel PCA 写成凸差 (DC) 问题再对偶化的统一框架

Kernel PCA (核主成分分析) 是无监督学习里最常用的工具之一：它把线性 PCA 搬到核诱导的特征空间里，从而能刻画数据中的非线性结构。但它有一个顽固的老问题：标准做法要对 $n \times n$ 的 Gram 矩阵做一次奇异值分解 (SVD)，代价是 $O(n^3)$ 。数据量稍微上一点规模，这一步就慢得让人难以接受。

更麻烦的是，人们常常还想要 Kernel PCA 的一些「加强版」：对离群点更稳健 (robust) 的版本，或者能得到稀疏 (sparse) 解的版本。可长期以来，每加一个性质，往往就要重新设计一套专门的优化问题和启发式权重，最后攒出一堆彼此无关的方法，而不是一个统一的框架。

这篇来自 KU Leuven、发表在 ICML 2023 的论文给出了一个漂亮的答案：把 Kernel PCA 重写成一个「凸差」(difference of convex functions, DC) 问题，再对它做对偶化。仅此一步，就同时换来了三样东西：一个绕开昂贵 SVD 的梯度求解器，一个能自由切换稳健 / 稀疏目标的模块化框架，以及在真实数据集上可观的加速。

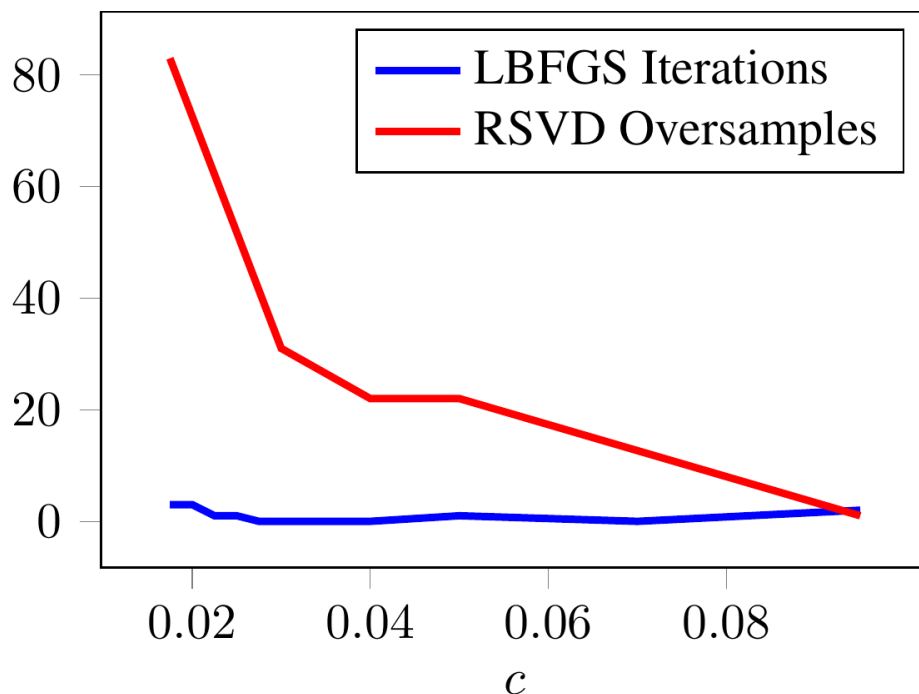


图 1 特征谱衰减对两种求解器的影响：横轴 c 越大表示 Gram 矩阵 G 的特征谱衰减越快。随着谱衰减变慢 (c 变小)，随机化 SVD (红色, RSVD Oversamples) 需要的过采样数急剧上升，而本文基于 L-BFGS 的求解器 (蓝色, LBFGS Iterations) 迭代次数几乎不变，这正是它在真实数据上稳定占优的原因。

论文链接：<https://arxiv.org/abs/2306.05815>

代码链接：<https://github.com/taralloc/dc-kpca>

问题：经典 Kernel PCA 卡在哪

经典 Kernel PCA 的求解，本质上是对 $n \times n$ 的 Gram 矩阵求前若干个主特征方向，也就是一次 $O(n^3)$ 的 SVD。 n 上万之后，这一步既慢又吃内存。论文在计时实验里也印证了这一点：在 RCV1 ($n=20242$) 和 CIFAR-10 ($n=60000$) 上，把完整 SVD 算到机器精度需要超过 30 分钟，直接超时。

另一条线索来自「加强版」的需求。要让 Kernel PCA 对离群点稳健，或者让解变稀疏，过去通常靠单独设计的损失或加权技巧。这些做法各自为战，缺少一个能把它们统一起来解释和求解的视角。论文想同时解决这两件事：既要快，又要能优雅地容纳稳健和稀疏。

核心想法：写成凸差问题，再对偶化

出发点是一个已经被优化领域研究得很透的观察：Kernel PCA 可以写成在正交约束（Stiefel 流形）下的方差最大化，而这恰好属于「凸差」问题的家族，即目标形如两个凸函数之差 $\inf g(W) - f(\Gamma W)$ 。此前也有人用 DC 视角研究 PCA，但只覆盖线性 PCA，或者只处理正交约束消失的第一个主成分。这篇论文把它推广到了非线性、多主成分的核心情形。

关键的一步是「对偶化」。原问题的变量 W 活在可能是无穷维的特征空间里，没法直接做梯度下降；论文的一般对偶命题把它转成一个对偶问题，对偶变量 H 是一个有限的 $n \times s$ 矩阵（ s 是要保留的主成分数），天然适合梯度法。Stiefel 流形的指示函数在对偶里变成一个核范数（nuclear norm）项 $\pi(H) = \text{Tr}\sqrt{H^T G H}$ ，它的梯度只需要对一个很小的 $s \times s$ 矩阵做 SVD，而不是那个庞大的 $n \times n$ 矩阵。 $O(n^3)$ 就此降成每步 $O(s^3)$ ，这就是加速的来源。

标准 Kernel PCA 的对偶问题用 L-BFGS 求解即可。要引入稳健或稀疏，只需在目标里再加一个 Moreau 包络（Moreau envelope）项 $\Psi_\star(H)$ ，然后换用凸差算法（DCA）求解，并对 Huber 损失和 ε -不敏感损失给出闭式的近端算子（proximal operator）。整个过程始终只碰 $s \times s$ 的小 SVD，从不触及 $n \times n$ 的大 SVD。换句话说，稳健和稀疏不是三套不同的方法，而是同一个对偶框架里换一个目标函数的结果。

第一件事：更快

效率实验横跨合成与真实数据：Synth 1 ($n=7000$)、Protein ($n=14895$)、RCV1 ($n=20242$) 和 CIFAR-10 ($n=60000$)。所有求解器用同一个相对对偶代价残差作为统一停机准则，在 $\delta=10^{-2}$ 和 $\delta=10^{-4}$ 两档容差下比较，计时结果对 5 次运行取平均。下表是较松容差 $\delta=10^{-2}$ 下的训练时间（秒）与相对随机化 SVD 的加速倍数。

表 1 $\delta=10^{-2}$ 下各求解器的 Kernel PCA 训练时间（秒）与相对 RSVD 的加速倍数；「>30 min」表示完整 SVD 在该数据集上超时。数据摘自论文 Table 2。

数据集	n	SVD	Lanczos	RSVD	本文	加速
Synth 1	7000	96.73	0.85	1.97	0.53	3.72x
Protein	14895	868.64	3.46	6.70	1.07	6.25x
RCV1	20242	>30 min	6.04	12.50	2.12	5.90x
CIFAR-10	60000	>30 min	48.10	123.89	13.51	9.17x

结论很干净：在 $\delta=10^{-2}$ 下，本文求解器在每个任务上都快过完整 SVD、Lanczos 和随机化 SVD，且至少比 RSVD 快 3 倍，在 CIFAR-10 上达到 9.17× 的峰值加速，而此时完整 SVD 连 30 分钟都跑不完。

图 1 进一步解释了为什么这种优势在真实数据上稳得住。随机化 SVD 的精度依赖 Gram 矩阵特征谱的衰减速度：谱衰减慢（这在真实数据里很常见）时，RSVD 要靠大幅增加过采样数才能达到同样的精度，成本随之飙升；而本文求解器的迭代次数对这种谱变化几乎无感。

那 s 变大会怎样？图 2 在 $n=15000$ 的随机数据上考察主成分数 s 对训练时间的影响。在很大范围内，L-BFGS 都比 RSVD 更快；只有当 s 非常大时，每步都要做的那个 $s \times s$ SVD 才开始拖累它，两条曲线才逐渐靠拢。对典型的降维任务（ s 取几十量级），这个拐点远在实用区间之外。

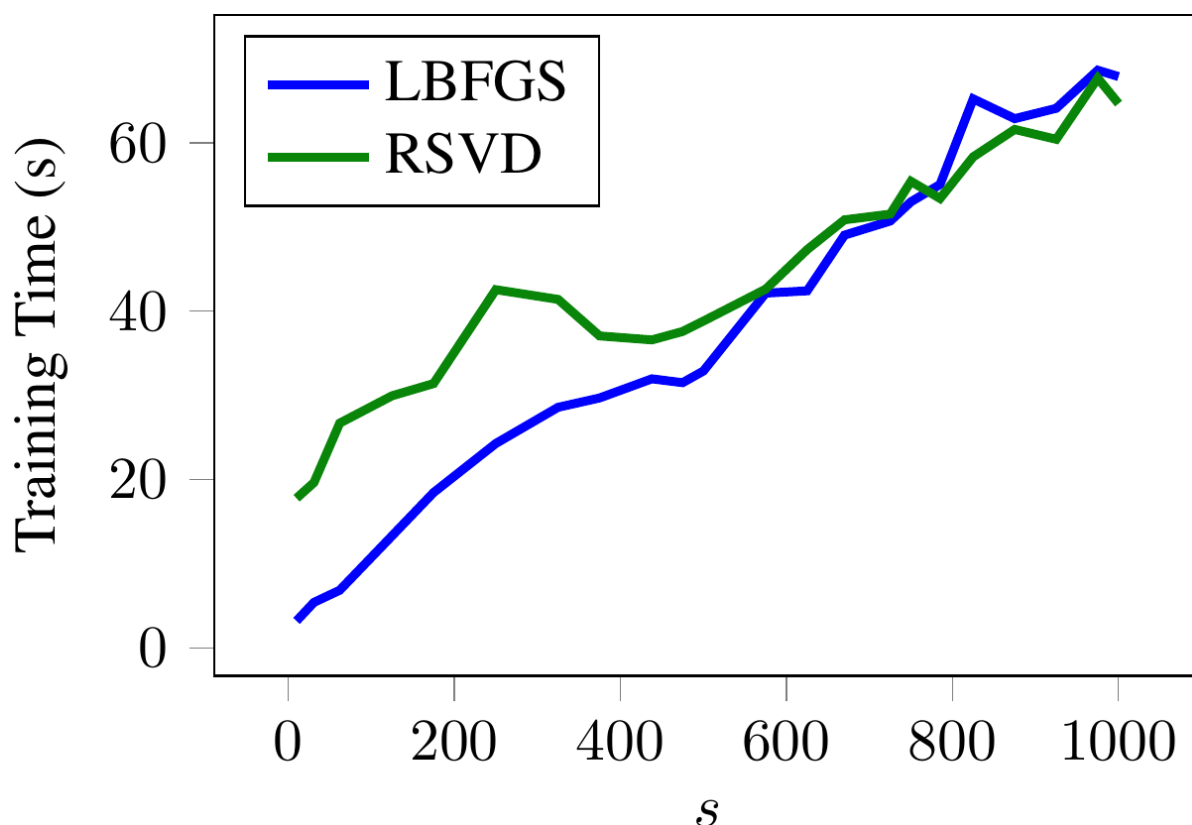


图 2 主成分数 s 对训练时间的影响（ $n=15000$ 随机数据）： s 增大时两种方法耗时都上升，但在很宽的范围内 L-BFGS（蓝）始终快于 RSVD（绿）；只有 s 非常大时，每步的 $s \times s$ SVD 才让二者接近。

第二件事：更稳健

稳健性来自把目标里的平方损失换成 Huber 损失，这在对偶框架里对应加一个特定的 Moreau 包络项。论文考虑两种 Huber 变体： H^1_κ 与 H^2_κ （分别是对 ℓ^1 和逐样本 ℓ^2 范数取 Moreau 包络），二者都用同一个 DC 算法优化。实验在被乘性高斯离群点污染的 Iris 数据集上进行，用高斯核，看重构 MSE。

图 3 扫描 Huber 参数 κ ，把两条 Huber 曲线和红色虚线（普通平方损失 $\frac{1}{2}\|\cdot\|^2$ ）作对比。可以看到，在合适的 κ 范围内， H^1_κ 和 H^2_κ 的 MSE 都能压到平方损失基线之下，说明稳健性确实被框架诱导出来了。论文 Table 4 给出的具体数值也印证了这一点：在污染水平 $\tau=10$ 时， H^2_κ 的 MSE 为 6.83，明显低于平方损失的 7.59。

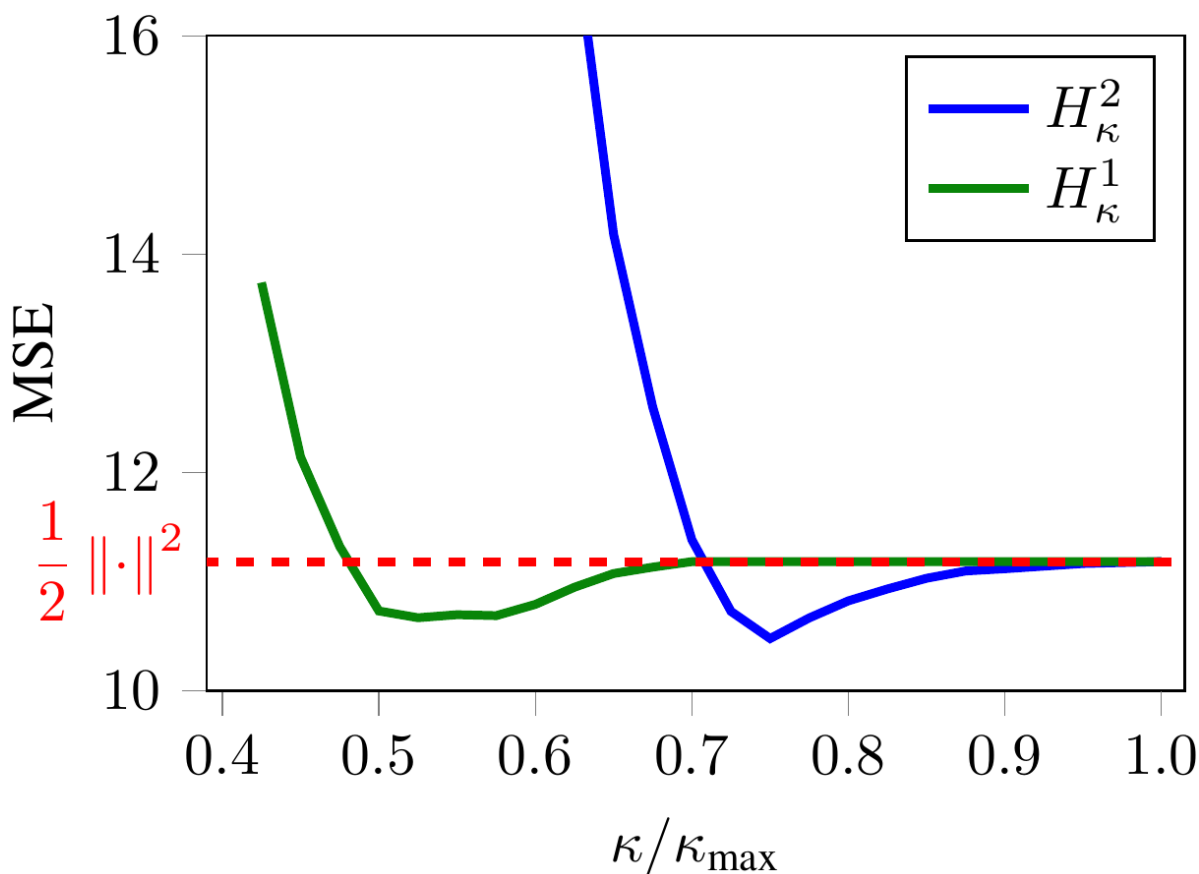


图 3 Huber 损失的稳健性（受污染的 Iris，高斯核）：横轴为归一化的 Huber 参数 $\kappa/\kappa_{\max}$ ，纵轴为 MSE。红色虚线是平方损失基线；在合适的 κ 下， H^1_κ （绿）与 H^2_κ （蓝）都能把 MSE 压到基线以下。

第三件事：更稀疏

同一个框架换成 ϵ -不敏感损失（ ϵ -insensitive loss），就能得到稀疏的 Kernel PCA：许多对偶系数被精确地压成零，从而只用一小部分样本来表示主成分。代价自然是重构精度会有所损失，问题在于这个折中是否划算。

图 4 画出 ℓ^2_ϵ 损失下的重构误差比（相对稠密解）随稀疏度（被置零系数的比例）的变化，不同颜色对应不同的主成分数 s 。曲线整体平缓：即便稀疏度很高，重构误差比也只是温和上升，说明可以用相当稀疏的解，换来只略微变差的重构质量，得到一个可控、可解释的精度—稀疏度折中。

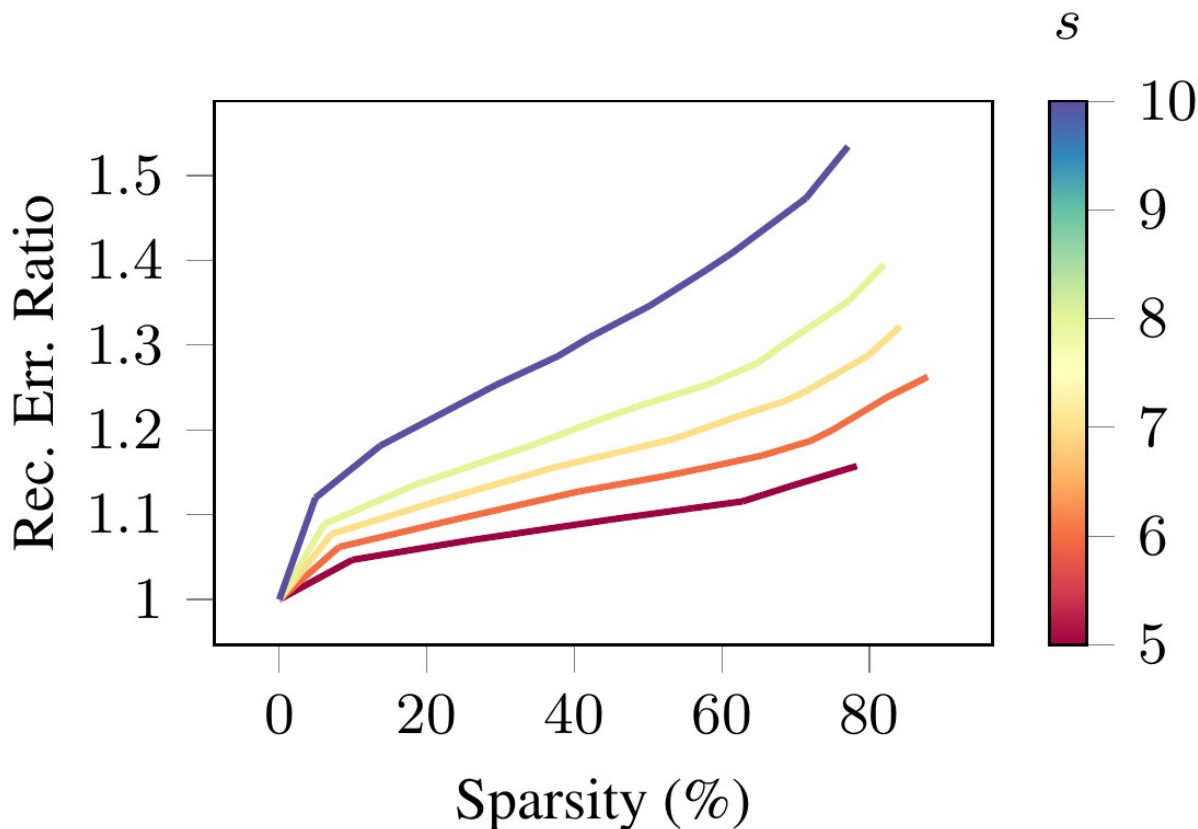


图 4 稀疏 Kernel PCA 的精度—稀疏度折中 (ℓ^2_ϵ 损失)：横轴为稀疏度（被置零系数比例），纵轴为相对稠密解的重构误差比，颜色表示主成分数 s 。曲线平缓上升，说明高稀疏度下重构质量只是温和下降。

小结：一个视角，三个方向

这篇论文的价值，在于用一次视角的切换换来了三个方向上的收益。把 Kernel PCA 写成凸差问题并转到对偶之后，你得到的是一个绕开 $O(n^3)$ SVD、比随机化 SVD 最高快约 9 倍的求解器；与此同时，只要在目标里换一个 Moreau 包络项，就能按需索取稳健（Huber 损失）或稀疏（ ϵ -不敏感损失），而不必另起炉灶。

还有一个容易被忽略但很关键的性质：对偶变量 H 永远是有限维的 ($n \times s$)，所以这个框架连经典基于 SVD 的 Kernel PCA 无法直接处理的无穷维特征映射也能覆盖。当然，边界也很清楚：当主成分数 s 非常大时，每步 $s \times s$ SVD 的开销会逐渐吃掉速度优势；稀疏则是用重构精度换来的，需要按任务权衡。但作为一个把「快、稳、稀疏」统一进同一套对偶优化语言的框架，它给出的是一种干净且可扩展的思路。